S NO	Program Title	Page No.	Date
1	Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.		
2	Make the above web application responsive web application using Bootstrap framework.		
3	Use JavaScript for doing client – side validation of the pages implemented in experiment 1 and experiment 2.		
4	Explore the features of ES6 like arrow functions, callbacks, promises, async/await. Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.		
5	Create an xml for the bookstore. Validate the same using both DTD and XSD		
6	Maintaining the transactional history of any user is very important. Explore the various session tracking mechanism (Cookies, HTTP Session)		
7	Create a custom server using http module and explore the other modules of Node JS like OS, path, event		
8	Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman)		

9	For the above application create authorized end points using JWT (JSON Web Token).		
10	Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages.		

WEEK-1: Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.

Here's how you can build a responsive shopping cart web application with registration, login, catalog, and cart pages using HTML5, CSS3 features (Flex and Grid), and basic interactivity.

---

1. Project Setup

Folder Structure scss

Copy code shopping-cart/

```
|
├── index.html        (Home/Login Page)
├── register.html     (Registration Page)
├── catalog.html      (Catalog Page)
├── cart.html         (Cart Page)
├── css/
|   └── styles.css    (Global styles)
└── js/
    └── app.js        (For basic interactivity)
```

---

2. Registration Page (register.html)

HTML: html

Copy code

```
<!DOCTYPE html>  <html
lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Register</title>
 <link rel="stylesheet" href="css/styles.css">
</head>
<body>
 <header class="header">
  <h1>Shopping Cart</h1>
 </header>
 <main class="form-container">
  <form class="form" id="registerForm">
   <h2>Register</h2>
   <label for="username">Username:</label>
   <input type="text" id="username" name="username" required>

   <label for="email">Email:</label>
   <input type="email" id="email" name="email" required>

   <label for="password">Password:</label>
   <input type="password" id="password" name="password" required>

   <button type="submit">Register</button>
```

```
  </form>
 </main>
</body>
</html>
CSS: css
Copy code
body {
margin:
0;
fontfamily:
Arial,
sans-
serif;
lineheight:
1.6;
}

.header {   backgroundcolor:
#333;
 color: #fff;   textalign:
center;
padding: 1rem;
}

.form-container {
```

```
 display: flex;
justifycontent: center;
alignitems: center;  height:
100vh;  padding: 1rem;
}

.form {  background: #f4f4f4;  padding:
2rem;  border-radius: 8px;
boxshadow: 0 0 10px rgba(0, 0, 0, 0.1);
max-width: 400px;   width: 100%;
}

label {
display: block;   margin:
1rem 0 0.5rem;
}

input {  width: 100%;
padding: 0.5rem;  margin-
bottom: 1rem;  border: 1px
solid #ddd;  border-radius:
4px;
}

button {
background: #333;
 color: #fff;
```

```
 border: none;  padding:
0.75rem;  cursor:
pointer;  width:
100%;  border-radius:
4px;
}

button:hover { background:
 #555;

}
```

---

3. Catalog Page (catalog.html)

HTML: html

Copy code

```
<!DOCTYPE html>  <html
lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Catalog</title>
 <link rel="stylesheet" href="css/styles.css">
</head>
<body>
 <header class="header">
  <h1>Product Catalog</h1>
 </header>
 <main class="catalog">
```

```
  <div class="product-card">
    <img src="https://via.placeholder.com/150" alt="Product 1">
    <h3>Product 1</h3>
    <p>$10.00</p>
    <button>Add to Cart</button>
  </div>
  <div class="product-card">
    <img src="https://via.placeholder.com/150" alt="Product 2">
    <h3>Product 2</h3>
    <p>$15.00</p>
    <button>Add to Cart</button>
  </div>
  <!-- Add more products -->
 </main>
</body>
</html>
```

CSS: css

Copy code

```
.catalog {  display: grid;  grid-template-columns:
repeat(auto-fit, minmax(200px, 1fr));  gap: 1rem;  padding:
2rem;
}

.product-card {  border:
1px  solid  #ddd;
borderradius:  8px;
overflow: hidden;  text-
```

```
align: center;    padding:
1rem;
}

.product-card img {   width:
100%; height:
auto;
}

.product-card h3 {   margin:
1rem 0 0.5rem;
}

.product-card p {   margin:
0.5rem 0;   fontweight:
bold;
}

.product-card button {   background:
#333;
  color: #fff;
  border: none;   padding:
0.5rem;   cursor: pointer;
border-radius: 4px;
}

.product-card button:hover {   background:
#555;
```

}

---

4. Cart Page (cart.html)

HTML: html

Copy code

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Cart</title>
 <link rel="stylesheet" href="css/styles.css">
</head>
<body>
 <header class="header">
  <h1>Your Cart</h1>
 </header>
 <main class="cart-container">
  <div class="cart-item">
   <h3>Product 1</h3>
   <p>Quantity: 1</p>
   <p>Price: $10.00</p>
   <button>Remove</button>
  </div>
  <!-- Add more cart items dynamically -->
 </main>
</body>
</html>
```

CSS: css

Copy code

.cart-container { display:

flex;

```
 flex-direction: column;  padding:
2rem;
}

.cart-item {  border: 1px
solid                  #ddd;
borderradius:        8px;
padding:
1rem;  margin-bottom:
1rem;
}

.cart-item button {
background: #f44336;
 color: #fff;
 border: none;  padding:
0.5rem;  cursor: pointer;
border-radius: 4px;
}

.cart-item button:hover {  background:
#d32f2f;
}
```

---

5. JavaScript Interactivity

JS for Adding Items to Cart (app.js):

javascript

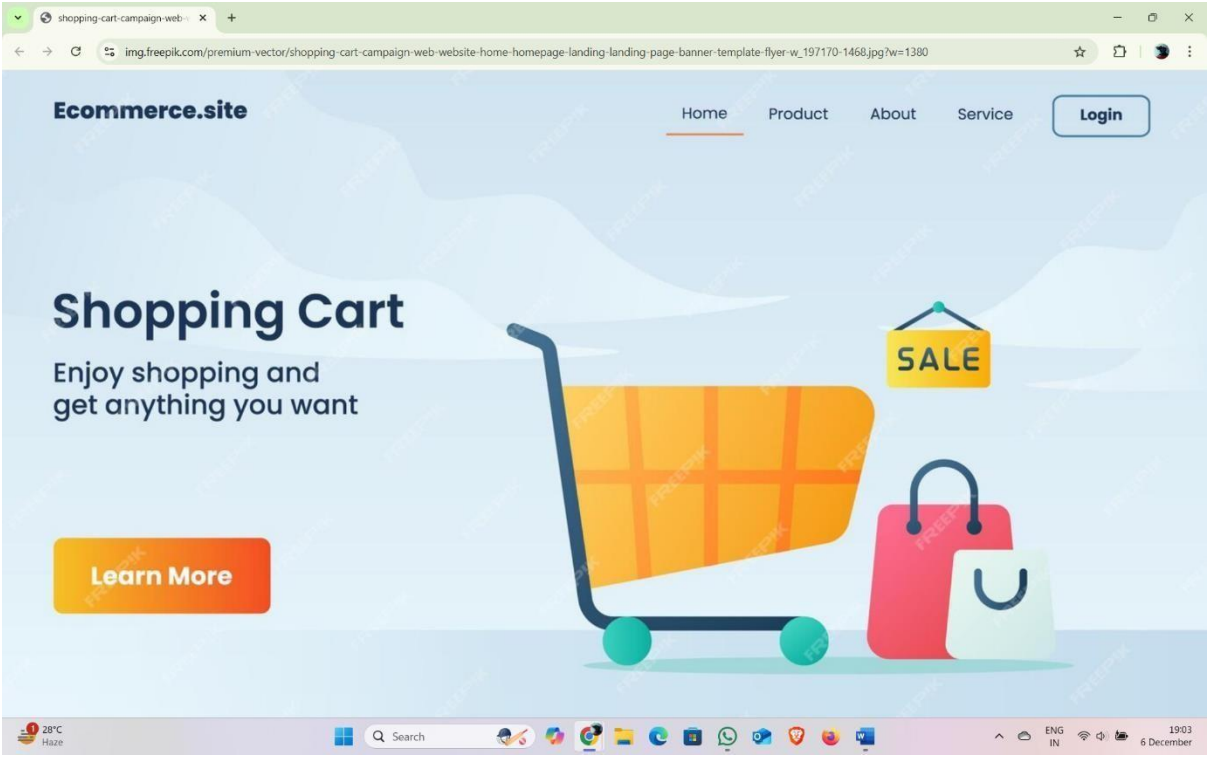
Copy code

```
document.addEventListener('DOMContentLoaded', () => {   const buttons
= document.querySelectorAll('.product-card button');

  buttons.forEach((button) => {
button.addEventListener('click', () => {      alert('Item
added to cart!');
    });
  });
});
```

This structure can be expanded further with dynamic functionality using JavaScript or backend integration for a complete web application.

Output:



WEEK -2: Make the above web application responsive web application using Bootstrap framework.

To make the shopping cart web application responsive using the Bootstrap framework, you can replace custom CSS (like display: flex and display: grid) with Bootstrap classes. Below is how you can implement the registration, login, catalog, and cart pages with Bootstrap for responsiveness.

---

1. Add Bootstrap to Your Project

Include the Bootstrap CSS and JS in your HTML file headers. Use the Bootstrap CDN for simplicity: html

Copy code

```
<!-- Include Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha3/dist/css/bootstrap.min.css" rel="stylesheet">

<!-- Include Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha3/dist/js/bootstrap.bundle.min.js"></script>
```

---

2. Update Registration Page (register.html)

HTML: html

Copy code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Register</title>
```

```
 <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
 <header class="bg-dark text-white text-center py-3">
  <h1>Shopping Cart</h1>
 </header>
 <main class="d-flex justify-content-center align-items-center vh-100">
  <form class="p-4 bg-light rounded shadow" style="width: 100%; max-width: 400px;">
   <h2 class="text-center mb-4">Register</h2>
   <div class="mb-3">
    <label for="username" class="form-label">Username:</label>
    <input type="text" class="form-control" id="username" name="username"
required>
   </div>
   <div class="mb-3">
    <label for="email" class="form-label">Email:</label>
    <input type="email" class="form-control" id="email" name="email" required>
   </div>
   <div class="mb-3">
    <label for="password" class="form-label">Password:</label>
    <input type="password" class="form-control" id="password" name="password"
required>
   </div>
   <button type="submit" class="btn btn-primary w-100">Register</button>
  </form>
 </main>
</body>
```

```
</html>
```

---

3. Update Catalog Page (catalog.html)

HTML: html

Copy code

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Catalog</title>
 <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
 <header class="bg-dark text-white text-center py-3">
  <h1>Product Catalog</h1>
 </header>
 <main class="container my-5">
  <div class="row g-4">
   <!-- Product Card 1 -->
   <div class="col-sm-6 col-md-4 col-lg-3">
    <div class="card">
     <img src="https://via.placeholder.com/150" class="card-img-top" alt="Product 1">
     <div class="card-body text-center">
      <h5 class="card-title">Product 1</h5>
      <p class="card-text">$10.00</p>
      <button class="btn btn-primary">Add to Cart</button>
```

```
        </div>
      </div>
    </div>
    <!-- Add more product cards as needed -->
  </div>
 </main>
</body>
</html>
```

---

4. Update Cart Page (cart.html)

HTML: html

Copy code

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Cart</title>
 <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
 <header class="bg-dark text-white text-center py-3">
  <h1>Your Cart</h1>
 </header>
 <main class="container my-5">
  <ul class="list-group">
   <!-- Cart Item -->
```

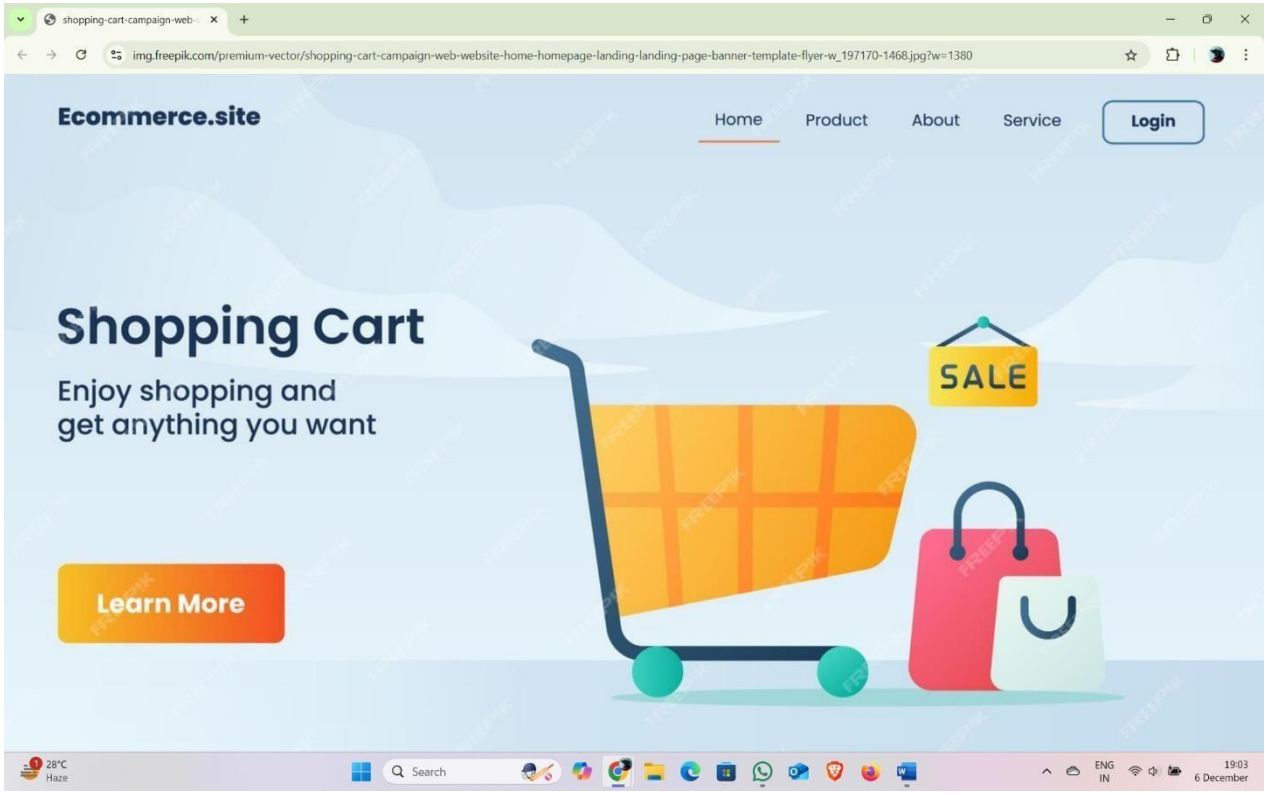
```
    <li class="list-group-item d-flex justify-content-between align-items-center">
      <div>
        <h5>Product 1</h5>
        <p>Quantity: 1</p>
        <p>Price: $10.00</p>
      </div>
      <button class="btn btn-danger">Remove</button>
    </li>
    <!-- Add more cart items dynamically -->
  </ul>
 </main>
</body>
</html>
```

---

5. Benefits of Using Bootstrap

1. **Responsive Grid System:**
    - **Replace custom flexbox or grid layouts with container, row, and col-* classes for better responsiveness.**
2. **Pre-styled Components:**
    - **Use Bootstrap buttons, forms, and cards to simplify styling.**
3. **Mobile-first Design:**
    - **Bootstrap automatically adjusts layouts for different screen sizes using col-, col-sm-, col-md-, and other classes.**
4. **Quick Prototyping:**
    - **Minimal CSS is required; most functionality is handled by Bootstrap's built-in styles.**

This Bootstrap-based implementation ensures that your shopping cart application is fully responsive, mobile-friendly, and modern in appearance. It can be easily enhanced with JavaScript or backend integration for dynamic functionality.



The above is the desired output and obtained output which is same as the first week experiment output.

WEEK-3: Use the site Validation using JavaScript for the Shopping Cart webpage described in Week 1 and Week 2.

Here's how you can add **client-side validation using JavaScript** to the pages of your shopping cart application. This will ensure that form inputs are properly validated before submission.

---

1. Add JavaScript File

Include a single JavaScript file (js/validation.js) for validation logic across the application. Link it to all your HTML files:

html

Copy code

<script src="js/validation.js"></script>

---

2. Validation for the Registration Page JavaScript (validation.js):

javascript Copy

code

document.addEventListener('DOMContentLoaded', () => { const registerForm = document.getElementById('registerForm');

if (registerForm) {
registerForm.addEventListener('submit', (e) => {
e.preventDefault(); // Prevent default submission
const username = document.getElementById('username').value.trim(); const email = document.getElementById('email').value.trim(); const password = document.getElementById('password').value.trim(); // Validation Rules if (username === "") { alert('Username cannot be empty');

```
      return;
    }
    if (!validateEmail(email)) {
      alert('Please enter a valid email address');
return;
    }
    if (password.length < 6) {
      alert('Password must be at least 6 characters long');
return;
    }

    alert('Registration successful!');
    registerForm.submit(); // Submit the form programmatically
  });
 }
});

// Helper Function for Email Validation function validateEmail(email)
{
 const re = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6}$/;
return re.test(email);
}
```

Registration Page (register.html):

Make sure the form includes an id for JavaScript to target. html

Copy code

```
<form id="registerForm">
  <!-- Registration form fields -->
</form>
```

---

3. Validation for the Login Page JavaScript:

Extend the same validation.js file: javascript

Copy

code

```
 const loginForm = document.getElementById('loginForm');

 if (loginForm) {
   loginForm.addEventListener('submit', (e) => {
     e.preventDefault();
     const email = document.getElementById('email').value.trim();
const password = document.getElementById('password').value.trim();

     if (!validateEmail(email)) {
       alert('Please enter a valid email address');
return;
     }
     if (password === "") {
       alert('Password cannot be empty');
return;
     }

     alert('Login successful!');
```

```
loginForm.submit();
   });
 }
```

Login Page (login.html): html

Copy code

```
<form id="loginForm">
  <!-- Login form fields -->
</form>
```

---

4. Additional Interactivity on Catalog and Cart Pages

While catalog and cart pages typically don't require form submissions, you can validate actions like adding items to the cart or removing them.

JavaScript for Catalog (Adding to Cart):

javascript Copy

code

```
const catalogButtons = document.querySelectorAll('.product-card button');

catalogButtons.forEach((button) => {
button.addEventListener('click', () => {
alert('Item added to cart!');
  });
});
```

JavaScript for Cart (Removing Items):

javascript Copy

code

```
const removeButtons = document.querySelectorAll('.cart-item button');
```

```
removeButtons.forEach((button) => {
button.addEventListener('click', (e) => {
   e.target.closest('.cart-item').remove();
alert('Item removed from cart!');
  });
});
```

---

5. Advantages of Client-Side Validation

1. **Improves User Experience**: ○ Provides immediate feedback without reloading the page.
2. **Reduces Server Load**:
    - Filters out invalid data before sending requests to the server.
3. **Customizable**:
    - Easily extendable for additional form validations.

---

6. Testing Validation

- **For Registration**:
    - Test with empty fields, invalid email formats, and passwords shorter than 6 characters.
- **For Login**: ○ Test with empty email/password and invalid email formats.
- **For Catalog/Cart**:
    - Ensure buttons work as expected and no console errors appear.

By adding this client-side validation, your shopping cart application becomes more robust and user-friendly!

WEEK-4: Explore the features of ES6 like arrow functions,callbacks,promises,async/await.Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.

To create an application that reads weather information from OpenWeatherMap API and displays it in a graph, we'll use ES6 features such as arrow functions, promises, async/await, and a charting library like Chart.js for visual representation.

Steps to Implement

2. **Setup an HTML file with a form for user input (e.g., city name).**
3. **Use OpenWeatherMap's API to fetch weather data.**
4. **Process the data to extract relevant information (e.g., temperature, dates).**
5. **Display the data using Chart.js.**

---

1. Prerequisites

- **Sign up at OpenWeatherMap and get your free API key.**
- **Include Chart.js in your HTML file.**

Include Chart.js:

html

Copy code

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

---

2. HTML Structure html

Copy code

<!DOCTYPE html>

<html lang="en">

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Weather App</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container my-5">
    <h1 class="text-center">Weather Information</h1>
    <form id="weatherForm" class="mb-4">
      <div class="input-group">
        <input type="text" id="cityInput" class="form-control" placeholder="Enter city
name" required>
        <button type="submit" class="btn btn-primary">Get Weather</button>
      </div>
    </form>
    <canvas id="weatherChart" width="400" height="200"></canvas>   </div>

  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

---

3. JavaScript (app.js) Using

ES6 Features javascript

Copy code

```
// OpenWeatherMap API Key const
```

```
API_KEY = 'your_api_key_here'; //
Fetch weather data using async/await
const fetchWeatherData
= async (city) => {
 try {
   const response = await fetch(

`https://api.openweathermap.org/data/2.5/forecast?q=${city}&units=metric&appid
=${API_KEY}`
   );
   if (!response.ok) throw new Error('City not found');
const data = await response.json();    return data;   } catch
(error) {    console.error(error);    alert(error.message);
 }
};

// Process the weather data to extract temperatures and dates const
processWeatherData = (data) => {
 const temperatures = data.list.slice(0, 8).map((entry) => entry.main.temp); // Next
24 hours   const labels = data.list.slice(0, 8).map((entry) =>    new
Date(entry.dt_txt).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
 );
 return { temperatures, labels };
};

// Render the chart using Chart.js const
renderChart = (labels, data) => {   const ctx
=
document.getElementById('weatherCh
```

```
art').getContext('2d');   new Chart(ctx, {
   type: 'line',    data:
{      labels:
labels,
datasets: [
      {
        label: 'Temperature (°C)',          data: data,
borderColor: 'rgba(75, 192, 192, 1)',          backgroundColor:
'rgba(75, 192, 192, 0.2)',          borderWidth: 2,
        fill: true,
      },
    ],
   },
   options: {
responsive: true,
    plugins: {        legend:
{
position: 'top',
      },
    },
   },
 });
};

// Handle form submission
document.getElementById('weatherForm').addEventListener('submit', async (event)
=> {
event.preventDefault();
```

```
 const city = document.getElementById('cityInput').value.trim();
 if (city === '') {
  alert('Please enter a city name');    return;
 }

 const weatherData = await fetchWeatherData(city); // Async call   if
(weatherData) {    const { temperatures, labels } =
processWeatherData(weatherData);    renderChart(labels, temperatures);
 }
});
```

---

4. Explanation of ES6 Features

1. **Arrow Functions:**
    - **Used for cleaner syntax in functions like processWeatherData and event listeners.**

```
javascript Copy code const processWeatherData
= (data) => {
 // Arrow function
};
```

2. **Promises:**
    - **fetch API returns a promise, allowing us to handle asynchronous operations.**

```
javascript Copy code
fetch(url)
 .then((response) => response.json())   .catch((error)
=> console.error(error));
```

3. **Async/Await:**
    - **Simplifies asynchronous code for fetching weather data (fetchWeatherData).**

javascript Copy code const

fetchWeatherData = async (city) => { const

response = await fetch(url); return

response.json();

};

4. **Destructuring:** ○ **Used to extract data like temperatures and labels.**

javascript Copy code

const { temperatures, labels } = processWeatherData(weatherData);

---

5. Example Output

1. **Form: User enters a city name.**
2. **Graph: A line chart displaying temperatures over the next 24 hours, dynamically generated based on the city's weather data.**

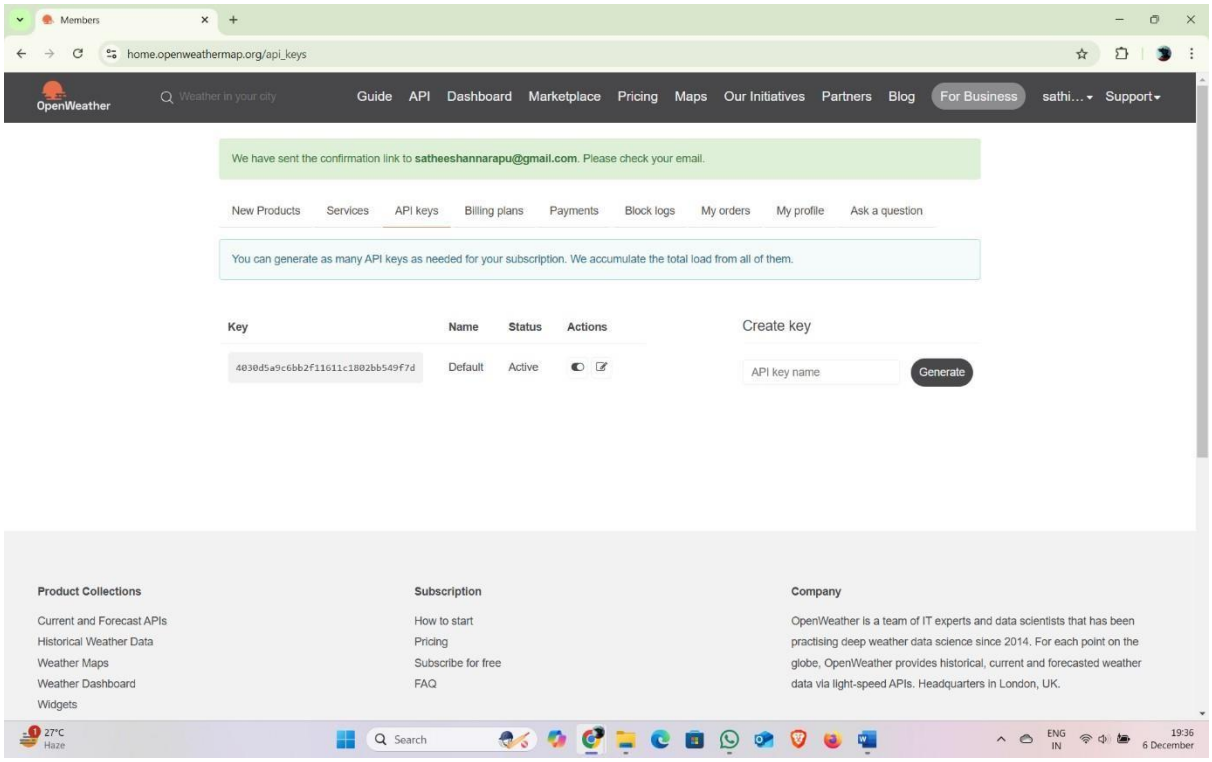
---

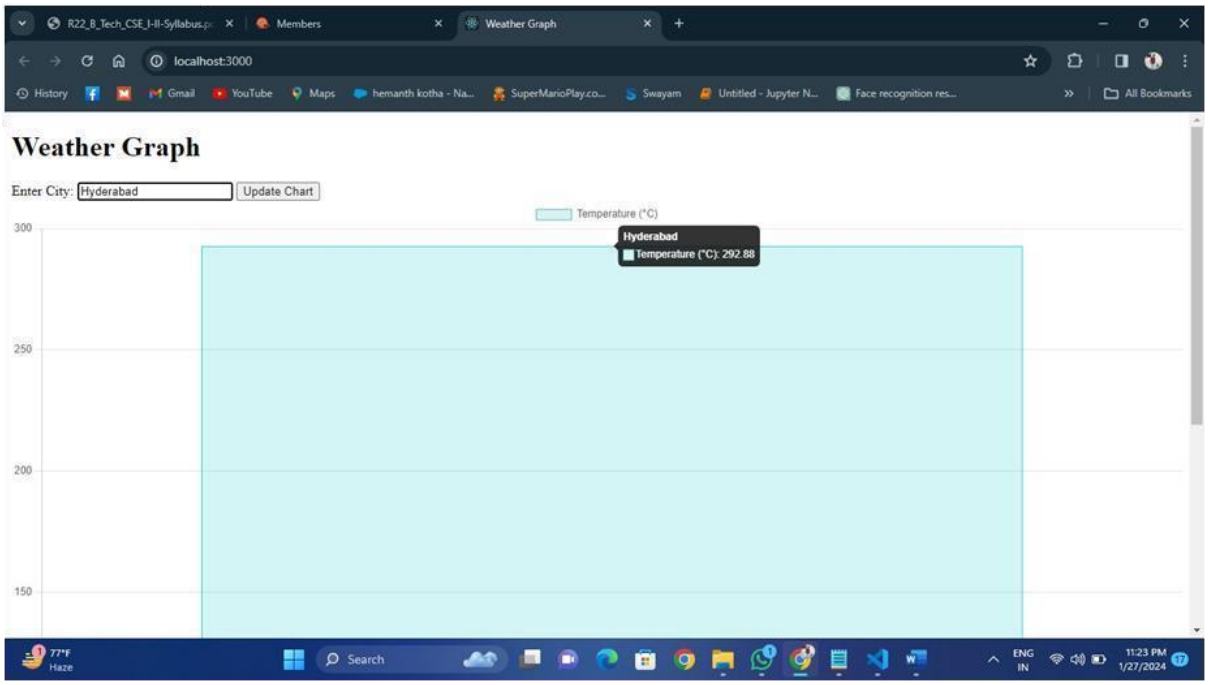
6. Advantages

- **Dynamic Updates:** ○ **Fetches live weather data.**
- **Responsiveness:**
  - **The app adapts to different screen sizes with Bootstrap and Chart.js.**
- **Modern JavaScript Features:**
  - **Uses ES6 for cleaner, more maintainable code.**

By combining ES6 features and Chart.js, this application effectively demonstrates clientside interactivity and modern development practices!

Output:


Members
home.openweathermap.org/api_keys
OpenWeather
Weather in your city
Guide API Dashboard Marketplace Pricing Maps Our Initiatives Partners Blog For Business sathi... Support
We have sent the confirmation link to satheeshannarapu@gmail.com. Please check your email.
New Products Services API keys Billing plans Payments Block logs My orders My profile Ask a question
You can generate as many API keys as needed for your subscription. We accumulate the total load from all of them.
Key Name Status Actions
4030d5a9c6bb2f11611c1802bb549f7d Default Active
Create key
API key name
Generate
Product Collections
Current and Forecast APIs
Historical Weather Data
Weather Maps
Weather Dashboard
Widgets
Subscription
How to start
Pricing
Subscribe for free
FAQ
Company
OpenWeather is a team of IT experts and data scientists that has been practising deep weather data science since 2014. For each point on the globe, OpenWeather provides historical, current and forecasted weather data via light-speed APIs. Headquarters in London, UK.
27°C
Haze
Search
ENG
IN
19:36
6 December



Week-6: Create an XML for a bookstore and validate it using DTD and XSD Validation methods.

Here's how to create an XML for a bookstore and validate it using DTD and XSD.

---

1. Create the XML Document bookstore.xml:

xml

Copy code

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookstore SYSTEM "bookstore.dtd">
<bookstore>
 <book id="b1">
  <title>Effective Java</title>
  <author>Joshua Bloch</author>
  <price>45.99</price>
  <year>2018</year>
 </book>
 <book id="b2">
  <title>Clean Code</title>
  <author>Robert C. Martin</author>
  <price>37.95</price>
  <year>2008</year>
 </book>
 <book id="b3">
  <title>The Pragmatic Programmer</title>
  <author>Andy Hunt, Dave Thomas</author>
  <price>42.50</price>
  <year>1999</year>
 </book>
</bookstore>
```

---

2. Create the DTD File bookstore.dtd:

```
dtd
Copy code
<!ELEMENT bookstore (book+)>
<!ELEMENT book (title, author, price, year)>
```

<!ATTLIST book id ID #REQUIRED>

<!ELEMENT title (#PCDATA)>

<!ELEMENT author (#PCDATA)>

<!ELEMENT price (#PCDATA)>

<!ELEMENT year (#PCDATA)>

Explanation:

- **<!ELEMENT bookstore>: Defines the root element containing one or more <book> elements.**
- **<!ELEMENT book>: A book contains <title>, <author>, <price>, and <year>.**
- **<!ATTLIST book id ID #REQUIRED>: Adds a required id attribute to the <book> element.**
- **<!ELEMENT title>: Allows character data (#PCDATA) inside the <title> tag.**

---

3. Validate XML with DTD

You can use any XML editor or online validator to validate bookstore.xml against bookstore.dtd. For example:

- **Online XML Validator with DTD**

---

4. Create the XSD File bookstore.xsd:

xml

Copy code

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
 <xs:element name="bookstore">
  <xs:complexType>
   <xs:sequence>
    <xs:element name="book" maxOccurs="unbounded">
     <xs:complexType>
      <xs:sequence>
```

```
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="price" type="xs:decimal"/>
        <xs:element name="year" type="xs:integer"/>
       </xs:sequence>
       <xs:attribute name="id" type="xs:ID" use="required"/>
      </xs:complexType>
     </xs:element>
    </xs:sequence>
   </xs:complexType>
  </xs:element>
</xs:schema>
```

---

5. Update XML to Reference XSD bookstore.xml

(with XSD): xml

Copy code

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="bookstore.xsd">
  <book id="b1">
    <title>Effective Java</title>
    <author>Joshua Bloch</author>
    <price>45.99</price>
    <year>2018</year>
  </book>
  <book id="b2">
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
```

```
  <price>37.95</price>
  <year>2008</year>
 </book>
 <book id="b3">
  <title>The Pragmatic Programmer</title>
  <author>Andy Hunt, Dave Thomas</author>
  <price>42.50</price>
  <year>1999</year>
 </book>
</bookstore>
```

---

6. Validate XML with XSD

You can validate this XML file against the XSD using an XML editor or an online tool like:

- **Free Online XSD Validator**

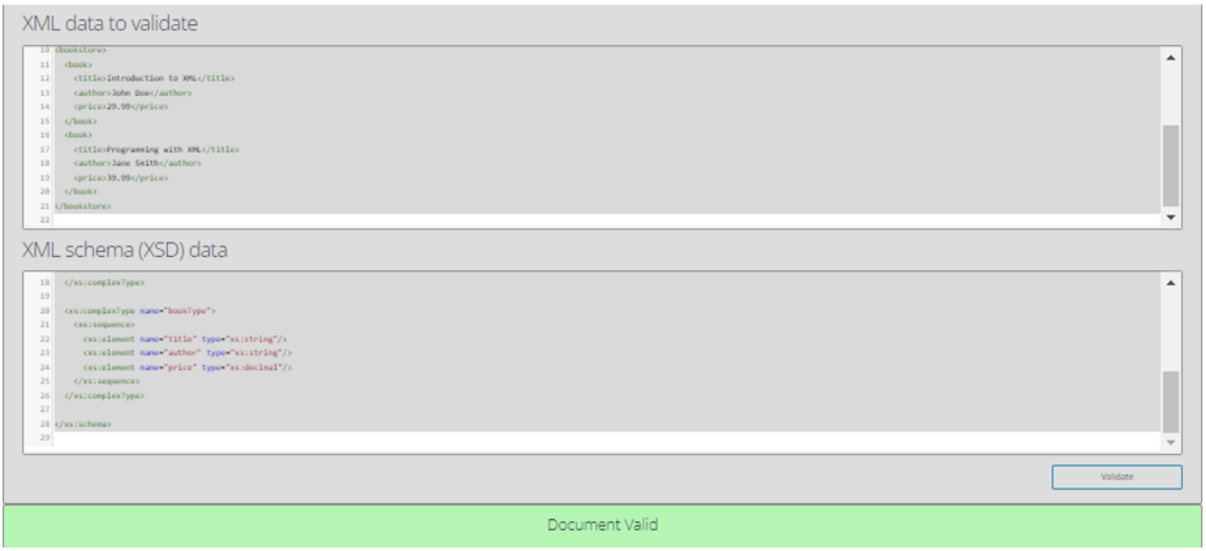
Summary

- **XML: Defines the bookstore data.**

- **DTD: Validates the structure and attributes of the XML.**

- **XSD: Provides more robust validation, including data types and constraints.**

Output:


XML data to validate
<bookstore>
<book>
<title>Introduction to XML</title>
<author>John Doe</author>
<price>29.99</price>
</book>
<book>
<title>Programming with XML</title>
<author>Jane Smith</author>
<price>39.99</price>
</book>
</bookstore>
XML schema (XSD) data
</xs:complexType>
<xs:complexType name="bookType">
<xs:sequence>
<xs:element name="title" type="xs:string"/>
<xs:element name="author" type="xs:string"/>
<xs:element name="price" type="xs:decimal"/>
</xs:sequence>
</xs:complexType>
</xs:schema>
Validate
Document Valid

WEEK-8: Maintaining the transactional history of any user is very important. Explore

the various session tracking mechanism (Cookies, HTTP Session).

Maintaining Transactional History Using Session Tracking Mechanisms

Maintaining the transactional history of a user involves tracking their activities and transactions during their interaction with a web application. Since HTTP is stateless, session tracking mechanisms are essential to preserve user state across requests. The two most common methods are Cookies and HTTP Sessions.

---

1. Cookies

What Are Cookies?

- **Cookies are small text files stored on the client's browser.**
- **They hold information in key-value pairs and are sent to the server with every HTTP request.**

Uses of Cookies

- **Storing user preferences, session IDs, or authentication tokens.**
- **Keeping track of items in a shopping cart.**
- **Remembering user login details for future visits.**

Key Features

- **Persistence: Cookies can be configured to persist across browser sessions by setting an expiration date.**
- **Access: Cookies can be read and written both on the server and client side.**
- **Size Limitation: Each cookie is limited to ~4KB.**

Advantages

- **Lightweight and easy to implement.**
- **Can persist data even after the session ends.**

Disadvantages

- **Limited storage capacity.**
- **Potential security vulnerabilities:**
  - **Susceptible to theft through Cross-Site Scripting (XSS).**
  - **Users can disable cookies in their browser.**
- **Can slow down requests due to repeated transmission with every HTTP call.**

Feature	Cookies	HTTP Sessions
Storage Location	Stored on the client-side (browser).	Stored on the server-side.
Data Size	Limited to 4KB per cookie.	Can store large amounts of data.
Persistence	Can persist across sessions.	Expires after the session ends or times out.

2. HTTP Sessions

What Are HTTP Sessions?

- **Sessions store data on the server side and are linked to a unique session ID stored on the client (typically in a cookie or URL).**
- **The server uses the session ID to retrieve data associated with the user's session.**

Uses of HTTP Sessions

- **Tracking logged-in user information.**
- **Managing sensitive data such as transaction history or cart details.**
- **Storing temporary user interactions during an active session.**

Key Features

- **Server-Side Storage: Data is stored securely on the server, making it less vulnerable to client-side attacks.**
- **State Management: Helps maintain state during a user’s interaction with the web application.**

Advantages

- **More secure since sensitive data is not stored on the client side.**
- **Can store larger amounts of data without affecting client performance.**

Disadvantages

- **Sessions consume server resources, which may increase with a large number of users.**
- **Sessions are usually temporary and expire after inactivity.**
- **Requires server-side infrastructure to manage session storage.**

Comparison: Cookies vs. HTTP Sessions

Less secure; data is exposed	**More secure; data is stored on the**

Security	**to the client.**	**server.**
Performance	**Minimal server load.**	**Increases server load as the number of sessions grows.**
Use Cases	**Storing user preferences, tokens.**	**Storing sensitive data like cart details or user history.**

---

3. Key Factors in Choosing a Session Tracking Mechanism

1. Security Requirements:

- **Use HTTP Sessions for sensitive data like transactional history or user authentication.**
- **Avoid storing sensitive data directly in cookies.**

2. Scalability:

- **Cookies are better for lightweight, scalable solutions since they don't consume server resources.**
- **Sessions can strain server resources in high-traffic applications.**

3. Persistence:

- **Use cookies if data needs to persist across browser sessions (e.g., "Remember Me" functionality).**
- **Sessions are suitable for temporary data that lasts only during the user's interaction.**

4. User Privacy:

- **Be mindful of privacy concerns and ensure compliance with data protection regulations (e.g., GDPR).**

---

4. Best Practices

- **For Cookies:**
  - **Use HttpOnly and Secure flags to protect cookies.**
  - **Avoid storing sensitive information directly in cookies.**
  - **Encrypt cookie data if necessary.**
- **For Sessions:**

- **Use HTTPS to secure the transmission of session IDs.**
- **Implement session timeouts to**

prevent unauthorized access. ○ Store session data in centralized stores (e.g., Redis) for distributed systems.

Real-World Applications

1. E-Commerce:

- **Cookies: Store user preferences, recently viewed items, or cart data.**
- **Sessions: Track logged-in user information and transactional history. 2. Banking Applications:**
- **Use sessions to securely track user activities, like money transfers or account changes.**

3. Social Media Platforms:

- **Combine cookies (for authentication tokens) and sessions (for user state and interactions).**

By combining the strengths of cookies and sessions where appropriate, web applications can provide seamless and secure user experiences.

WEEK-9 : Create a custom server using http module and explore the other modules of Node JS like OS, path, event.

Exploring Node.js Modules and Creating a Custom Server

Node.js provides a rich set of built-in modules to facilitate server-side development. These modules include http, os, path, and events. Below is an overview of these modules and how they can be explored through the creation of a simple custom server.

---

1. HTTP Module

The http module is used to create servers and handle HTTP requests and responses.

Key Concepts:

- **http.createServer(callback): Creates an HTTP server.**

- **req (Request): Represents the HTTP request from the client.**

- **res (Response): Represents the HTTP response that will be sent back to the client.**

How to Create a Custom Server Using the HTTP Module

To set up a simple server:

javascript Copy code const http =

require('http');

// Create the server const server = http.createServer((req, res)

=> { res.writeHead(200, { 'Content-Type': 'text/plain' });

res.write('Welcome to the Custom Server');

res.end();

});

// Set the server to listen on port 3000 server.listen(3000, () => {

console.log('Server is running at http://localhost:3000');

});

How This Works:

1. **http.createServer: Creates an HTTP server instance.**
2. **res.writeHead(200, { 'Content-Type': 'text/plain' }): Sets the HTTP status code and response header.**
3. **res.write('Welcome to the Custom Server'): Sends a simple response message to the client.**
4. **server.listen(3000): Makes the server listen on port 3000.**

When you visit http://localhost:3000, you'll see "Welcome to the Custom Server."

---

2. OS Module

The os module provides information about the operating system's underlying platform.

Commonly Used Methods in os Module

1. **os.platform(): Returns the operating system platform.**
2. **os.arch(): Returns the architecture of the operating system (e.g., 'x64').**
3. **os.cpus(): Returns an array of CPU core details.**
4. **os.hostname(): Fetches the hostname of the system.**
5. **os.uptime(): Returns the system uptime in seconds.**

Example Usage: javascript Copy

code const os = require('os');

console.log('Operating System:', os.platform()); console.log('System Architecture:', os.arch()); console.log('Number of CPUs:', os.cpus().length);

console.log('Hostname:', os.hostname());

console.log('System Uptime:', os.uptime());

---

3. Path Module

The path module helps work with file and directory paths. It ensures paths are correctly resolved regardless of the operating system.

Common Methods in path Module

1. **path.join(): Joins paths using the correct platform's separators.**
2. **path.resolve(): Resolves a sequence of paths into an absolute path.**
3. **path.basename(): Extracts the filename from a path.**
4. **path.dirname(): Gets the directory name from a path.**
5. **path.extname(): Returns the file extension.**

Example Usage:

javascript Copy code const path =

require('path');

console.log('Join Paths:', path.join('/users', 'admin', 'documents')); console.log('Resolve Path:', path.resolve(__dirname, 'example.txt'));
console.log('File Extension:', path.extname('/path/to/file.txt'));

---

4. Events Module

The events module provides a way to handle event-driven programming with custom events.

Key Concept:

- **EventEmitter: The core class used to handle custom events.**

Common Methods

1. **on(event, callback): Listens for a specific event.**
2. **emit(event, args): Trigger or dispatch a specific event.**
3. **once(event, callback): Listens for the event only once.**

Example Usage: javascript Copy code const EventEmitter = require('events'); class MyEmitter extends EventEmitter {}

// Create an instance of EventEmitter const myEmitter = new MyEmitter();

// Listen for an event myEmitter.on('event', (data) => {
console.log('Event triggered with data:', data);
});

// Emit the event

myEmitter.emit('event', { message: 'Hello, Node.js' });

Putting It All Together

Here's how you can create a custom server and use os, path, and events with it: javascript

Copy code const http = require('http'); const os = require('os'); const path = require('path');

const EventEmitter = require('events'); // Create an event emitter class MyEmitter extends

EventEmitter {} const myEmitter = new MyEmitter();

// Create the server const server = http.createServer((req, res)

=> { res.writeHead(200, { 'Content-Type': 'text/plain' });

const systemInfo = `

OS: ${os.platform()} ${os.arch()}

Hostname: ${os.hostname()}

System Uptime: ${os.uptime()} seconds

Path Example: ${path.join(__dirname, 'example.txt')}

`;

res.write('Welcome to the Custom Server!\n'); res.write(systemInfo);

// Trigger a custom event myEmitter.emit('data', { url:

```
req.url });


 res.end();
});

// Set up event listeners myEmitter.on('data', (data) => {
console.log('Received data request
for:', data.url); });


// Start the server server.listen(3000, () => {   console.log('Server is running
at http://localhost:3000');
});
```

How This Example Works

1. **HTTP Server: The server serves system information and responds to the client.**
2. **OS Module: Information like the platform, architecture, uptime, and hostname are retrieved using the os module.**
3. **Path Module: Demonstrates how paths are joined and resolved.**
4. **EventEmitter: Handles custom events and logs requests for URLs via myEmitter.emit.**

Summary of Modules

1. **http: Used to create the server and handle HTTP requests/responses.**
2. **os: Retrieves operating system details.**
3. **path: Manages and resolves file paths.**
4. **events: Handles event-driven programming with custom events.**

Key Takeaway

- **Node.js provides a modular approach to handle server creation, system information, file**

paths, and custom events.

- **Custom servers can integrate os, path, and events modules to build scalable, maintainable applications.**

Output:

```
PS D:\SAP\Custom Server> node server
OS Type: Windows_NT
OS Platform: win32
OS Architecture: x64
CPU Cores: 4
Current Working Directory: D:\SAP\Custom Server
Joined Path: D:\SAP\Custom Server\public\images
Custom Event Triggered: { message: 'Hello from custom event!' }
Server running at http://127.0.0.1:3000/
```

WEEK-10: Develop an express web application that can interact with REST API to perform CRUD

operations on student data. (Use Postman)

Developing an Express Web Application for CRUD Operations on Student Data

This application demonstrates how to build an Express.js RESTful API that performs CRUD (Create, Read, Update, Delete) operations on student data. We'll also explain how to interact with this API using **Postman**.

Setup and Requirements

1. **Node.js**: Ensure you have Node.js installed.
2. **Express.js**: Install Express for building the web application.
3. **Postman**: Use Postman to test the API endpoints.
4. **JSON File or Array**: Use a temporary in-memory data structure (array) for simplicity, or replace it with a database like MongoDB for production.

Steps to Build the Application

1. Initialize the Project

1. Open a terminal and create a new project directory.
2. Run the following commands: bash Copy code mkdir student-api cd student-api npm init -y npm install express bodyparser

---

2. Build the Express Application

Create a file named server.js and add the following code: javascript Copy code

```
const express =
require('express'); const bodyParser = require('body-parser');

const app = express(); const
PORT = 3000;

// Middleware  app.use(bodyParser.json());

// In-memory data store let students = [
  { id: 1, name: 'John Doe', age: 20 },
{ id: 2, name: 'Jane Smith', age: 22 },
];

// Routes

// Create a new student app.post('/students', (req, res)
=> {   const { id, name, age } = req.body;
```

```
 // Validation   if (!id || !name || !age) {     return res.status(400).send({ message:
'All fields are required.' });
 }
 // Check for duplicate ID   if (students.some(student => student.id
=== id)) {     return res.status(400).send({ message: 'Student ID already
exists.' });
 }

 const newStudent = { id, name, age };
students.push(newStudent);   res.status(201).send(newStudent);
});

// Get all students app.get('/students',
(req, res) => {   res.send(students);
});
// Get a single student by ID app.get('/students/:id', (req, res) => {   const student =
students.find(s => s.id === parseInt(req.params.id));   if (!student) {     return
res.status(404).send({ message: 'Student not found.' });
 }
 res.send(student);
});
// Update a student's information app.put('/students/:id', (req, res)
=> {   const student = students.find(s => s.id === parseInt(req.params.id));   if (!student)
{     return
res.status(404).send({ message: 'Student not found.' });
}

 const { name, age } = req.body;   if (name)
student.name = name;
if (age) student.age = age;

 res.send(student);
});

// Delete a student app.delete('/students/:id', (req, res) => {   const index =
students.findIndex(s => s.id === parseInt(req.params.id));   if
(index === -1) {     return res.status(404).send({ message: 'Student not found.' });
 }
```

```
const deletedStudent = students.splice(index, 1);
```

res.send(deletedStudent[0]);
});

// Start the server app.listen(PORT, () => { console.log(`Server is running at http://localhost:${PORT}`);
});

---

3. Testing the Application with Postman

Endpoints

1. **Create a Student (POST)** ○ **URL**: http://localhost:3000/students ○ **Body** (JSON):

json Copy
code
{
"id": 3,
"name": "Alice Brown",
"age": 21
}

○ **Expected Response**:

json Copy
code
{
"id": 3,
"name": "Alice Brown",
"age": 21
}

2. **Get All Students (GET)** ○ **URL**: http://localhost:3000/students ○ **Expected Response**:

json Copy
code
[
{ "id": 1, "name": "John Doe", "age": 20 },
{ "id": 2, "name": "Jane Smith", "age": 22 },
{ "id": 3, "name": "Alice Brown", "age": 21 }
]

3. **Get a Student by ID (GET)** ○ **URL**: http://localhost:3000/students/2 ○ **Expected Response**:

json
Copy code
{ "id": 2, "name": "Jane Smith", "age": 22 }

4. **Update a Student (PUT)** ○ **URL**:

http://localhost:3000/students/3 ○ **Body** (JSON):
json Copy
code
{
"name": "Alice Green",
"age": 23
}
○ **Expected Response**:
json
Copy code
{ "id": 3, "name": "Alice Green", "age": 23 }

5. **Delete a Student (DELETE)** ○ **URL**: http://localhost:3000/students/3 ○ **Expected Response**:

json
Copy code
{ "id": 3, "name": "Alice Green", "age": 23 }

---

4. Explanation of Components

1. **Middleware**:
○ body-parser: Parses JSON data in requests.
○ app.use(bodyParser.json()): Enables the server to read the request body.
2. **CRUD Operations**:
○ **Create**: Adds new student data to the in-memory store.
○ **Read**: Fetches all or specific student data. ○ **Update**: Modifies student data based on the provided ID.
○ **Delete**: Removes student data by ID.
3. **Error Handling**: ○ Returns appropriate HTTP status codes and messages for invalid requests.
4. **RESTful API Design**:
○ Organized into endpoints (/students, /students/:id) with corresponding HTTP methods.

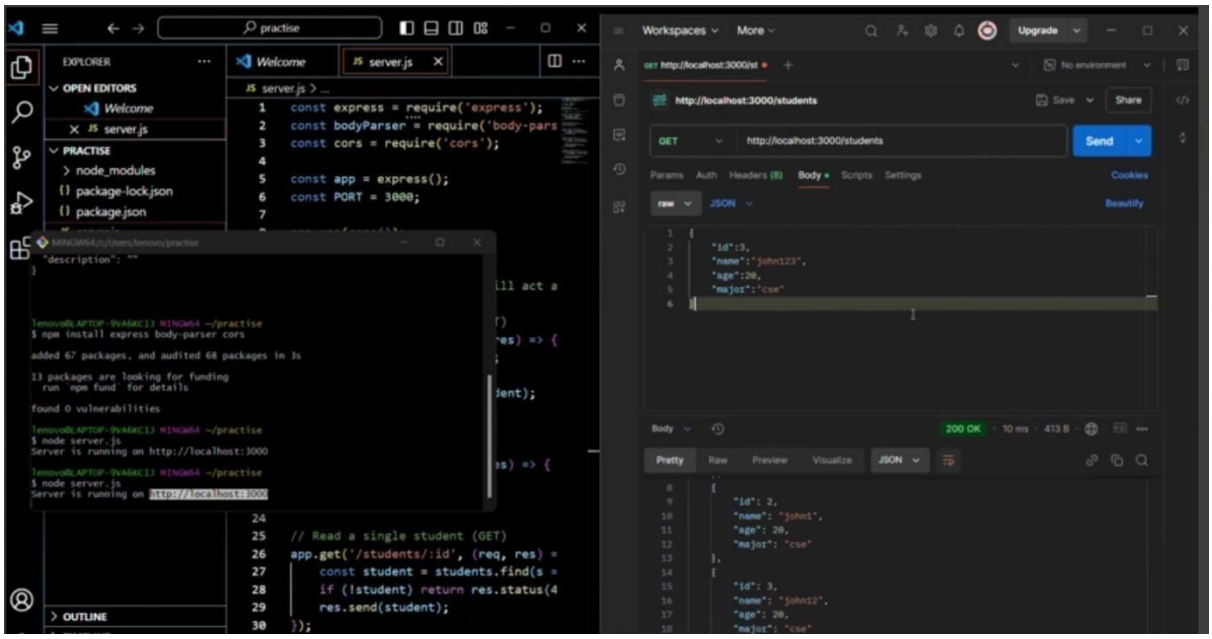
---

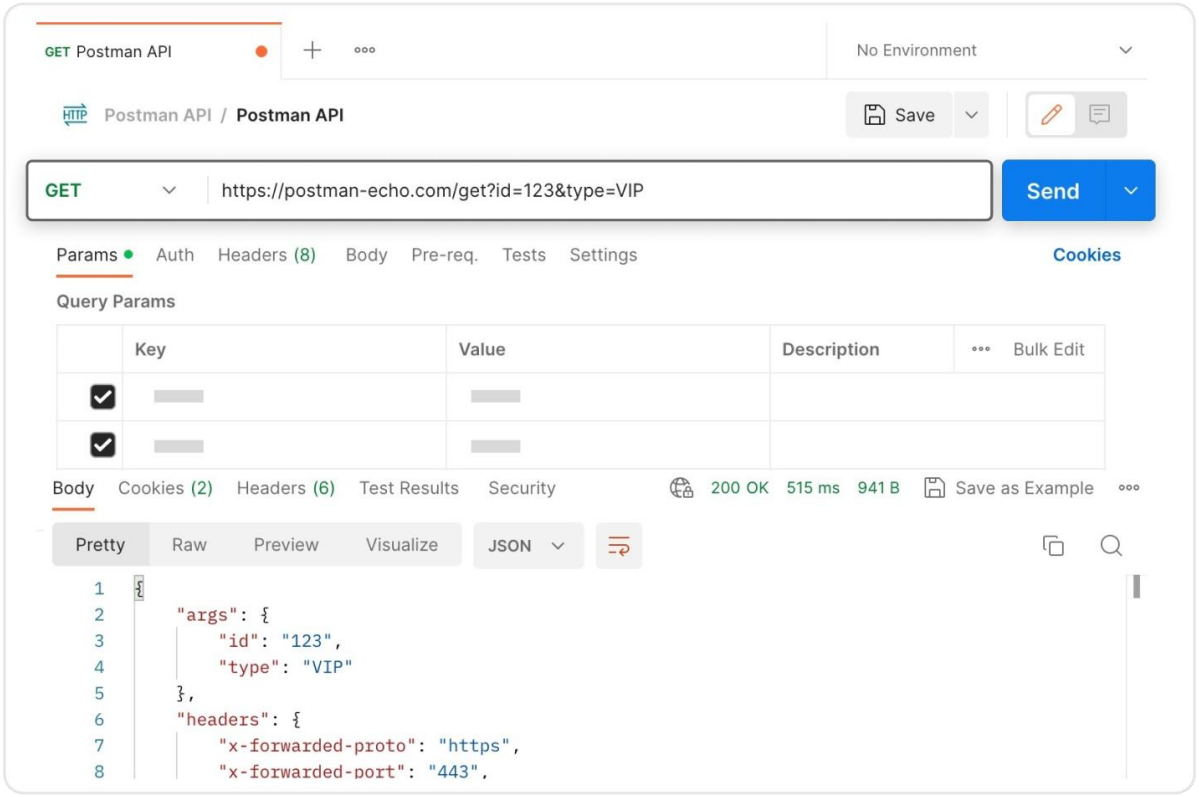
5. Enhancements for Production

1. **Database Integration**:
○ Replace the in-memory students array with a database like **MongoDB** or **MySQL**.
2. **Validation**: ○ Use libraries like Joi or express-validator for robust validation.
3. **Authentication**: ○ Add authentication mechanisms like **JWT** for secured access.
4. **Logging**:
○ Use logging libraries like morgan for better monitoring.

This Express application provides a structured way to manage student data and demonstrates the power of REST APIs. By using Postman, you can interact with the API to verify each functionality.

Output:




GET Postman API
No Environment
Postman API / Postman API
Save
GET
https://postman-echo.com/get?id=123&type=VIP
Send
Params
Auth
Headers (8)
Body
Pre-req.
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies (2)
Headers (6)
Test Results
Security
200 OK
515 ms
941 B
Save as Example
Pretty
Raw
Preview
Visualize
JSON
{
"args": {
"id": "123",
"type": "VIP"
},
"headers": {
"x-forwarded-proto": "https",
"x-forwarded-port": "443",

This is the Postman API which is used in this experiment.

WEEK-11: For the above application create authorized end points using JWT (JSON Web Token).

Securing Express API Endpoints with JWT Authentication

This section explains how to add JWT (JSON Web Token) authentication to the Express application we built earlier. JWT is a widely used standard for securely transmitting information between parties as a JSON object. It is commonly used to protect API endpoints.

Steps to Implement JWT Authentication

1. Install Required Dependencies

Run the following command to install necessary libraries:

bash Copy code npm install jsonwebtoken bcryptjs

- **jsonwebtoken: For creating and verifying JWTs.**
- **bcryptjs: For securely hashing and verifying passwords.**

---

2. Update the Application Code

Here's how you can integrate JWT into the application:

Step 1: Add a User Database

For simplicity, we'll use an in-memory array to store user credentials. Replace this with a database in production.

javascript Copy

code

```
let users = [
    { id: 1, username: 'admin', password: '$2a$10$E2pZvBtI8f/3lyOaVAF9H.djeNNM42.OFQESc5ChR5RJVVJc2Vb0m' } // Password: "admin123"
];
```

Step 2: Create Authentication Routes Add routes for registration and login. javascript Copy code const jwt = require('jsonwebtoken'); const bcrypt = require('bcryptjs');

```
const SECRET_KEY = 'your_secret_key';

// Register a new user app.post('/auth/register', async (req, res) => {  const { username, password } = req.body;
```

if (!username || !password) { return res.status(400).send({ message: 'Username and

password are required.' });
}

const hashedPassword = await bcrypt.hash(password, 10); users.push({ id:
users.length + 1, username, password: hashedPassword });

res.status(201).send({ message: 'User registered successfully.' });
});

// Login and generate a token app.post('/auth/login', async
(req, res) => { const { username, password } = req.body;

const user = users.find(u => u.username === username); if (!user) {
return res.status(401).send({ message: 'Invalid username or password.' });
}

const isPasswordValid = await bcrypt.compare(password, user.password);
if (!isPasswordValid) { return res.status(401).send({ message: 'Invalid username or
password.' });
}

const token = jwt.sign({ id: user.id, username: user.username }, SECRET_KEY, {
expiresIn: '1h' }); res.send({ token });
});

Step 3: Protect API Endpoints
Create middleware to verify JWT tokens and protect specific routes. javascript Copy code
const authenticate = (req,
res, next) => { const authHeader = req.headers.authorization;

if (!authHeader) { return res.status(401).send({ message: 'Authorization
token is required.' });
}

const token = authHeader.split(' ')[1];

try {
const decoded = jwt.verify(token, SECRET_KEY); req.user =
decoded; // Attach user info to request object next(); } catch (err)

```
{    return res.status(401).send({ message: 'Invalid or expired token.'
```

```
});
  }
};

// Example: Protect the GET /students endpoint app.get('/students',
authenticate, (req, res) => {   res.send(students);
});
```

How the JWT Workflow Operates

1. **User Registration:**
   - **The /auth/register endpoint allows new users to register with a username and password.**
   - **Passwords are securely hashed using bcrypt.**
2. **User Login:**
   - **The /auth/login endpoint verifies the username and password. ○ On successful authentication, a JWT is generated and returned to the client.**
3. **Protecting Routes:**
   - **The authenticate middleware checks for a valid JWT in the Authorization header.**
   - **If the token is valid, the request proceeds; otherwise, an error is returned.**

Testing the Application Using Postman

1. Register a User

- **URL: http://localhost:3000/auth/register • Method: POST**
- **Body: json Copy**

code

```
{
  "username": "admin",
  "password": "admin123"
}
```

- **Response:**

json Copy

code

```
{
  "message": "User registered successfully."
}
```

2. Login to Generate Token

- **URL: http://localhost:3000/auth/login • Method: POST**

- **Body: json Copy**

code

```
{
 "username": "admin",
 "password": "admin123"
}
```

- **Response:**

json Copy

code

```
{
 "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

3. Access Protected Route

- **URL: http://localhost:3000/students • Method: GET • Headers: ○ Authorization: Bearer <your_token>**
- **Response:**

json Copy code

```
[
 { "id": 1, "name": "John Doe", "age": 20 },
 { "id": 2, "name": "Jane Smith", "age": 22 }
]
```

4. Access Without Token

- **URL: http://localhost:3000/students • Method: GET • Headers: None**
- **Response:**

json Copy

code

```
{
 "message": "Authorization token is required."
}
```

Key Enhancements for Production

1. Environment Variables:

- **Store the SECRET_KEY securely using a .env file and load it with dotenv.**

2. Database Integration:

- **Replace the in-memory users and students arrays with a database (e.g., MongoDB or PostgreSQL).**

3. Token Expiry and Refresh:

- **Implement token expiration and refresh mechanisms for enhanced security.**

4. Role-Based Access Control:

- **Extend JWT to include roles (e.g., admin, student) and protect endpoints based on roles.**

By securing the Express API with JWT, we ensure that only authenticated users can access sensitive routes. This setup is scalable, secure, and forms the foundation for modern web applications.

WEEK-12: Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages.

Creating a React Application for a Student Management System
This guide explains how to build a React application with Registration, Login, Contact, and About pages, and implement routing to navigate between them using React Router.

Step 1: Initialize the React Application

1. Open a terminal and create a new React project: bash Copy code npx create-react-app student-management-system cd student-managementsystem npm install react-router-dom ○ react-router-dom: Provides routing capabilities for React applications. 2. Start the development server: bash Copy code npm start

---

Step 2: Project Structure
Organize the project with the following structure: css
Copy code src/

```
├── components/
│   ├── About.js
│   ├── Contact.js
│   ├── Login.js
│   ├── Registration.js
│   └── Navbar.js
├── App.js
├── index.js
```

Step 3: Implement Pages

1. Navbar Component
Create Navbar.js to provide navigation links: javascript Copy code

```
import React from 'react';
import { Link } from 'react-router-dom';

const Navbar = () => {
return (
  <nav>
    <ul style={{ display: 'flex', gap: '20px', listStyle: 'none' }}>
      <li><Link to="/">Home</Link></li>
      <li><Link to="/register">Register</Link></li>
      <li><Link to="/login">Login</Link></li>
      <li><Link to="/contact">Contact</Link></li>
      <li><Link to="/about">About</Link></li>
    </ul>
  </nav>
 );
};

export default Navbar;
```

---

2. Registration Page Create Registration.js:
javascript Copy code

```
import React, {
useState } from 'react';

const Registration = () => {   const [form, setForm] = useState({ username: '', email: '',
password: '' });

 const handleChange = (e) => {    setForm({ ...form, [e.target.name]:
e.target.value });
 };

 const handleSubmit = (e) => {
e.preventDefault();    console.log('Registration Data:', form);
 };

 return (
  <div>
   <h2>Registration</h2>
```

<form onSubmit={handleSubmit}>

```
                <input name="username" placeholder="Username"
onChange={handleChange} required />
      <input type="email" name="email" placeholder="Email" onChange={handleChange}
required />
      <input type="password" name="password" placeholder="Password"
onChange={handleChange} required />
      <button type="submit">Register</button>
    </form>
   </div>
 );
};

export default Registration;
```

---

3. Login Page Create Login.js:

```
javascript Copy code import React, {
useState } from 'react';

const Login = () => {   const [form, setForm] = useState({
email: '', password: '' });

 const handleChange = (e) => {    setForm({ ...form, [e.target.name]:
e.target.value });
 };

 const handleSubmit = (e) => {    e.preventDefault();
  console.log('Login Data:', form);
 };

 return (
  <div>
   <h2>Login</h2>
   <form onSubmit={handleSubmit}>
    <input type="email" name="email" placeholder="Email" onChange={handleChange}
required />
     <input type="password" name="password" placeholder="Password"
onChange={handleChange} required />
     <button type="submit">Login</button>
   </form>
```

```
</div>
```

```
  );
};

export default Login;
```

---

4. Contact Page Create Contact.js:

javascript Copy

code

```
import React from 'react';

const Contact = () => {
return (
   <div>
     <h2>Contact Us</h2>
     <p>Email: support@studentms.com</p>
     <p>Phone: +1-234-567-890</p>
   </div>
  );
};

export default Contact;
```

---

5. About Page Create About.js:

javascript Copy

code

```
import React from 'react';

const About = () => {
return (
   <div>
     <h2>About Us</h2>
     <p>The Student Management System helps manage student records efficiently.</p>
   </div>
  );
};

export default About;
```

Step 4: Configure Routing Modify App.js to include routing: javascript Copy code

```
import React from 'react'; import { BrowserRouter as
Router, Routes, Route } from 'react-router-dom'; import Navbar from
'./components/Navbar'; import Registration from
'./components/Registration'; import Login from './components/Login'; import Contact
from './components/Contact'; import About from
'./components/About';

const App = () => {
return (    <Router>
    <Navbar />
    <div style={{ padding: '20px' }}>
     <Routes>
       <Route path="/" element={<h2>Welcome to the Student Management
System</h2>} />
       <Route path="/register" element={<Registration />} />
       <Route path="/login" element={<Login />} />
       <Route path="/contact" element={<Contact />} />
       <Route path="/about" element={<About />} />
     </Routes>
    </div>
  </Router>
 );
};

export default App;
```

Step 5: Run the Application

1. **Start the application: bash Copy code npm start**
2. **Open your browser and navigate to http://localhost:3000. You can access:**

○ Home: http://localhost:3000/ ○ Registration: http://localhost:3000/register ○ Login: http://localhost:3000/login ○ Contact: http://localhost:3000/contact ○ About: http://localhost:3000/about

---

Enhancements

1. **Styling: Use CSS or Bootstrap for better UI design.**

2. State Management: Implement Redux or Context API for managing global state.

3. Form Validation: Use libraries like react-hook-form or Formik for better form validation.

4. API Integration: Connect the application to a backend for registration and login.

This application provides a basic structure for a Student Management System with routing and navigation. It can be extended with backend integration and additional features.

Output: